

Bootcamp IA Engineering

RAG Engineering

Recap Sprint 8 y 9 +
Sprint 10

Luis Ivan Umpire Alvarez

1. PREGUNTA A CHATGPT



TÚ



¿Cuál es la capital de España?

10:30 AM

CHATGPT



La capital de España es
Madrid.

10:30 AM ✓

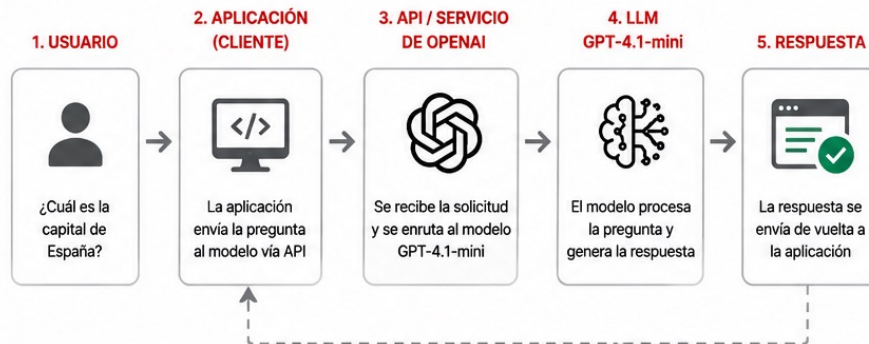


Estás usando ChatGPT (interfaz conversacional)

- Experiencia optimizada para conversar
- Historial de conversación
- Herramientas y funciones integradas
- Base de conocimiento del LLM

2. PREGUNTA ENVIÁNDOLA AL LLM GPT-4.1-mini

ARQUITECTURA DE ALTO NIVEL



RESPUESTA RECIBIDA POR LA APLICACIÓN

```
{
  "respuesta": "La capital de España es Madrid."
}
```



¿Qué está ocurriendo?

Tu aplicación envía la pregunta al modelo GPT-4.1-mini vía API. El modelo procesa la solicitud y devuelve la respuesta en formato texto (normalmente JSON), que tu aplicación puede usar.

CASO 1

CHATBOT BANCARIO

Un banco quiere desarrollar un asistente para atender consultas sobre tarjetas de crédito.



¿Cuál es la tasa de interés de la tarjeta Platinum?

¿Cuánto pago si retiro efectivo?

¿Qué seguros incluye mi tarjeta?

¿Cómo bloqueo mi tarjeta si la pierdo?



PROBLEMA: El LLM no conoce la información interna del banco.
No existe contexto.

CASO 1

CHATBOT BANCARIO

Un banco quiere desarrollar un asistente para atender consultas sobre tarjetas de crédito.



¿Cuál es la tasa de interés de la tarjeta Platinum?

¿Cuánto pago si retiro efectivo?

¿Qué seguros incluye mi tarjeta?

¿Cómo bloqueo mi tarjeta si la pierdo?



PROBLEMA: El LLM no conoce la información interna del banco.
No existe contexto.

← RAG

CASO 1

CHATBOT BANCARIO

Un banco quiere desarrollar un asistente para atender consultas sobre tarjetas de crédito.

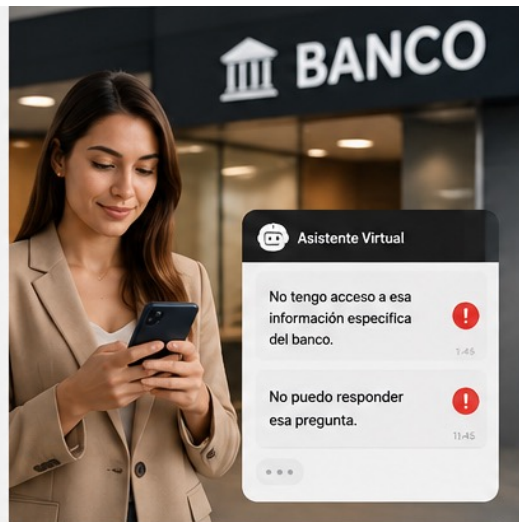


¿Cuál es la tasa de interés de la tarjeta Platinum?

¿Cuánto pago si retiro efectivo?

¿Qué seguros incluye mi tarjeta?

¿Cómo bloqueo mi tarjeta si la pierdo?



PROBLEMA: El LLM no conoce la información interna del banco.
No existe contexto.

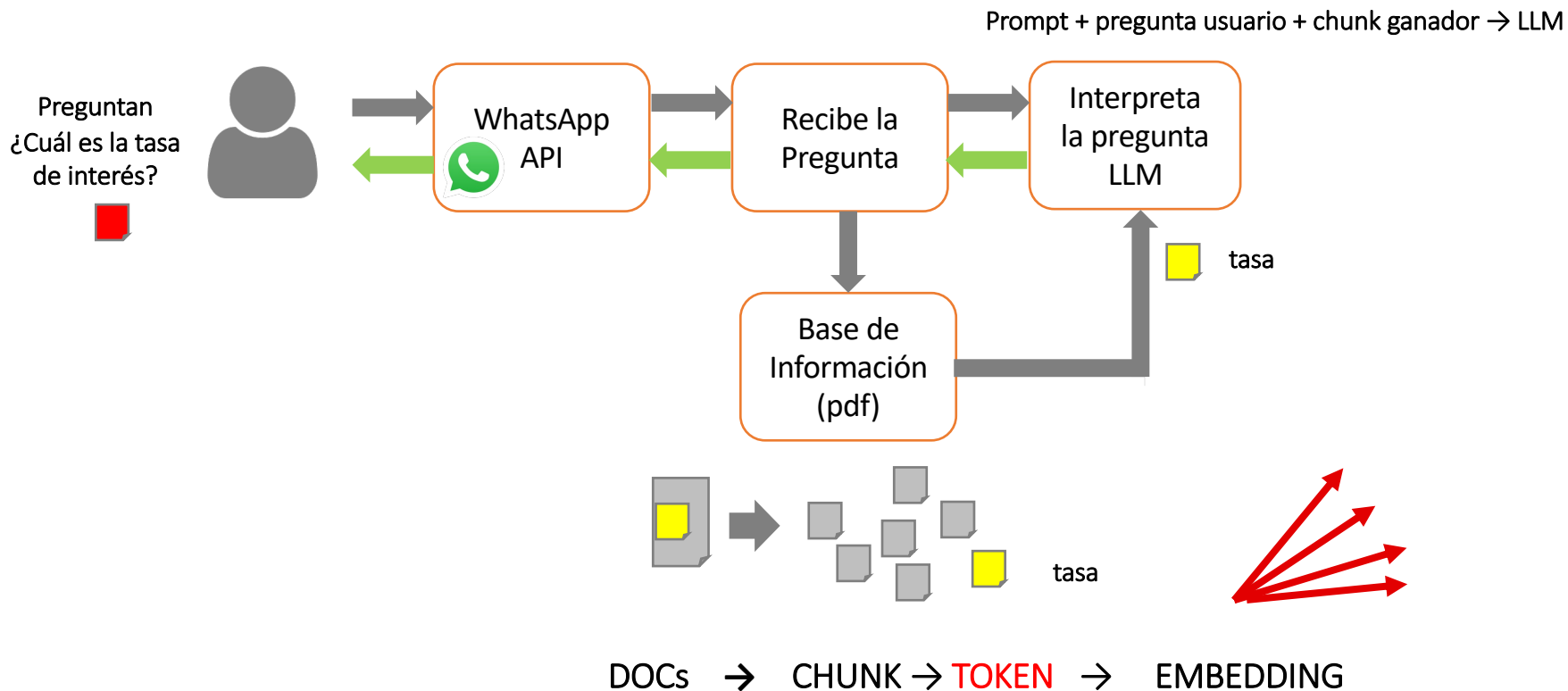


RAG

Dotar la base de conocimiento y los procesos al LLM para responder a usuarios preguntas relacionadas a las necesidades del negocio:
Ventas, Servicio al Cliente, etc.

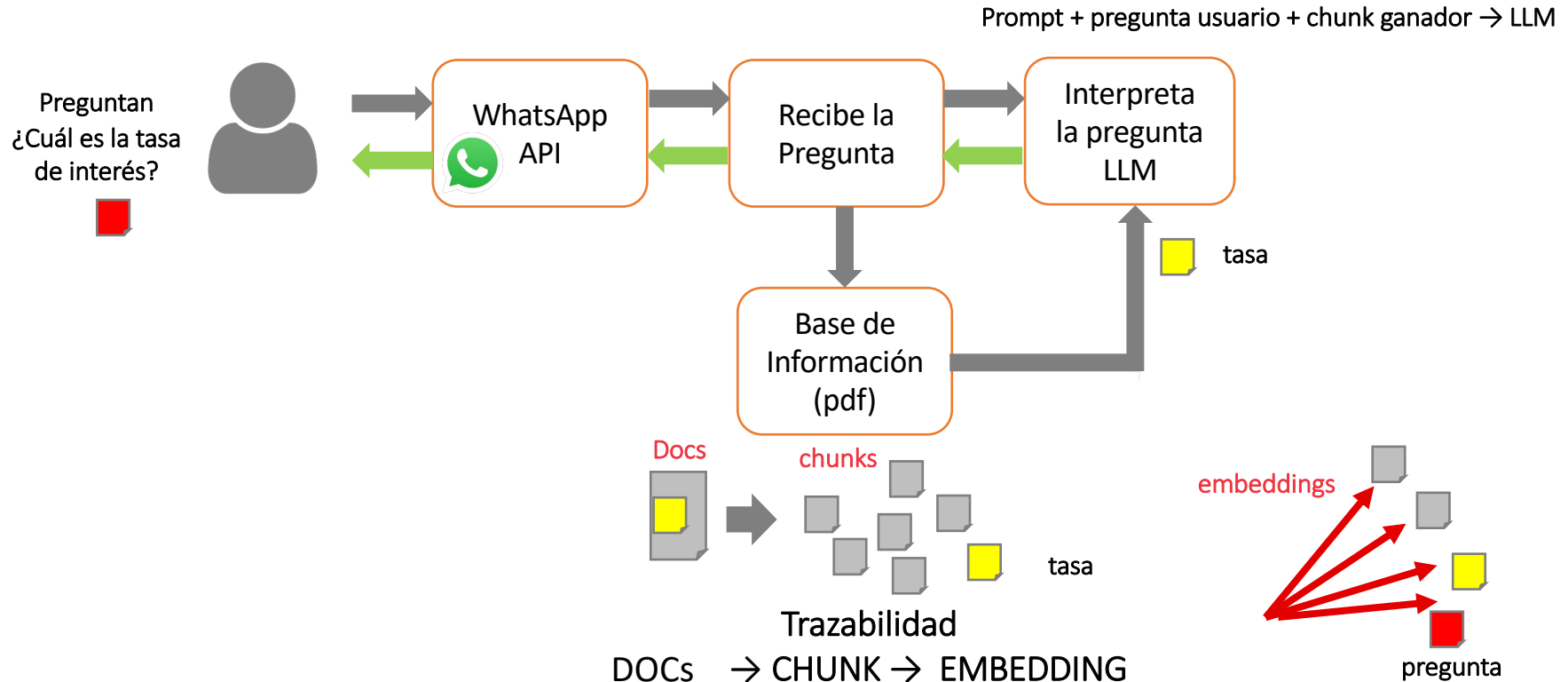
Ejemplo: Chatbot con IA

¿Qué problema observan? → Solución



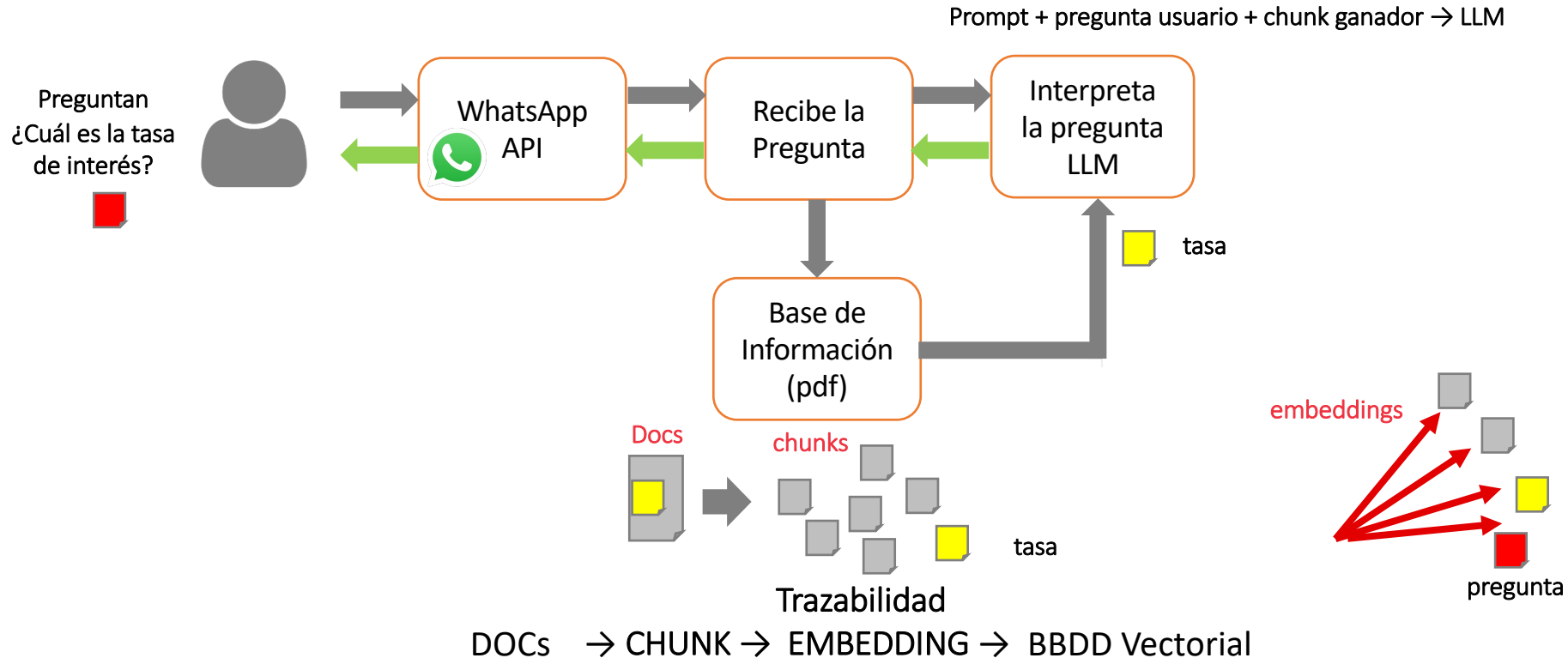
Ejemplo: Chatbot con IA

¿Qué problema observan? → Solución



Ejemplo: Chatbot con IA

¿Qué problema observan? → Solución



SPRINT 08 - SPRINT 09 -> SPRINT 10

SPRINT	¿QUÉ RESUELVE?		Artefactos
8	Base de conocimiento propia	Input:	Archivos fuente: csv, pdf, md, txt
		Process:	Modulos python: load.py, Clean.py, chunk.py, embed.py
		Output:	Chunks.json, embeddings.json
9	Recuperar los datos	Input:	Chunks.json, embeddings.json
		Process:	Busqueda semántica, vector store
		Output:	Bbdd vectorial: ChromaDB
10	LLM Generar respuesta	Input:	Bbdd vectorial, pregunta usuario
		Process:	Prompt, convertir la pregunta del usuario a embeddings, buscar los mejores embedding (top k)
		Output:	Respuesta al usuario.

Restricciones en el Prompt: no invente, solo use la info. Temperature baja

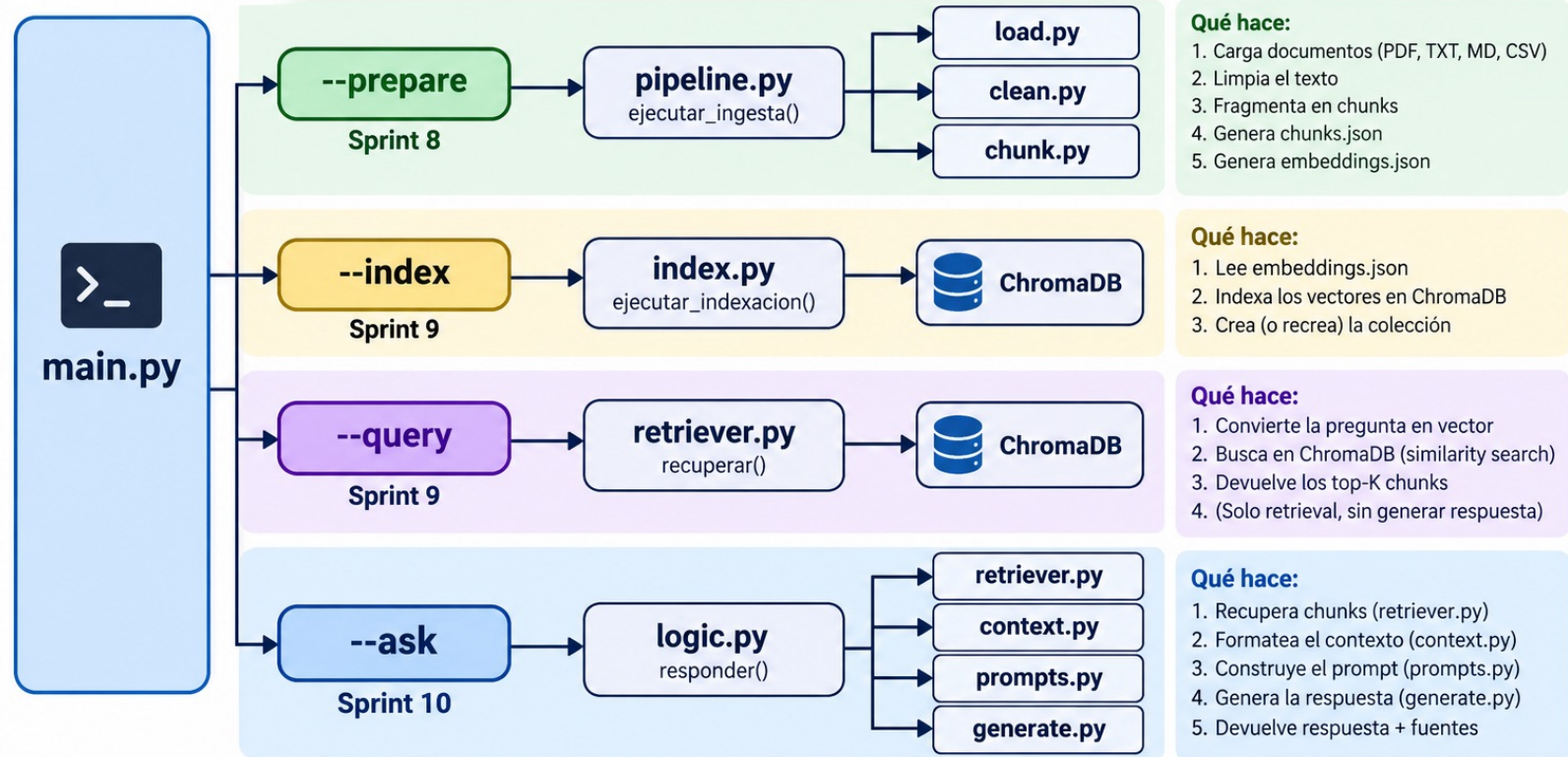
SPRINT 08 - SPRINT 09 -> SPRINT 10

Sprint	Qué resuelve	Artefactos
 Sprint 8	Preparamos la base de conocimiento	Input: archivos en diferentes formatos Proceso: procesamos en módulos de python Output: chunks.json y embeddings.json
 Sprint 9	Crear la capacidad de realizar búsqueda vectorial	Input: chunks.json y embeddings.json Proceso: módulos de python para indexar en ChromaDB y habilitar búsqueda vectorial Output: bbdd vectorial (ChromaDB)
 Sprint 10	Dar la capacidad al usuario que realice preguntas al LLM utilizando la base de conocimiento	Input: pregunta del usuario Proceso: convertir la pregunta en vector, utilizar procesos de búsqueda del Sprint 9 (retrieval) y construir el prompt para el LLM Output: respuesta al usuario

SPRINT 08 - SPRINT 09 -> SPRINT 10 : CÓDIGO

main.py — Orquestador de comandos (CLI)

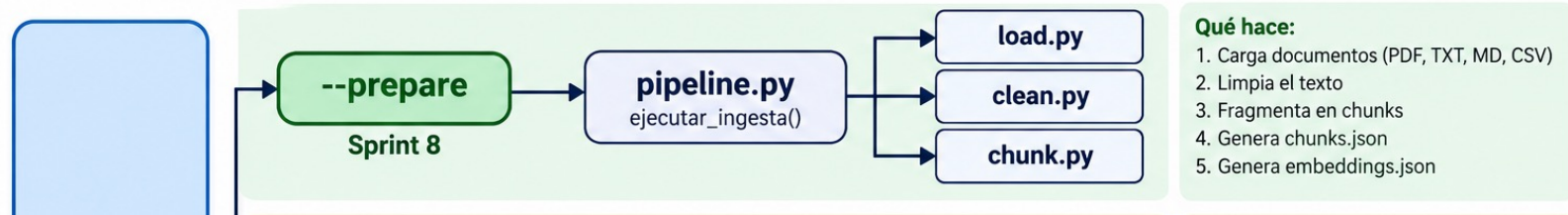
Recibe los argumentos de la terminal y ejecuta el módulo correspondiente.



SPRINT 08 - SPRINT 09 -> SPRINT 10 : CÓDIGO

main.py — Orquestador de comandos (CLI)

Recibe los argumentos de la terminal y ejecuta el módulo correspondiente.



```
% python main.py --prepare
```

```
def _cmd_prepare() -> None:
    ejecutar_ingesta()
    print()
    ejecutar_embeddings()
```

Documentos cargados: 45

Tras limpieza: 45

Chunks generados: 52

Embeddings: 52 chunks en 3754 ms (gemini-embedding-2)

Guardado: .../embeddings.json (3072 dimensiones)

SPRINT 08 - SPRINT 09 -> SPRINT 10 : CÓDIGO

main.py — Orquestador de comandos (CLI)

Recibe los argumentos de la terminal y ejecuta el módulo correspondiente.



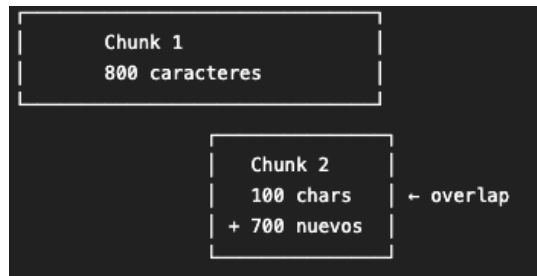
Chunk.py

```
def _crear_splitter() ->
RecursiveCharacterTextSplitter:
return RecursiveCharacterTextSplitter(
    chunk_size=CHUNK_SIZE,
    chunk_overlap=CHUNK_OVERLAP,
    length_function=len,
)
```

--- Ingesta y chunking (Sprint 8) ---

`CHUNK_SIZE = 800`

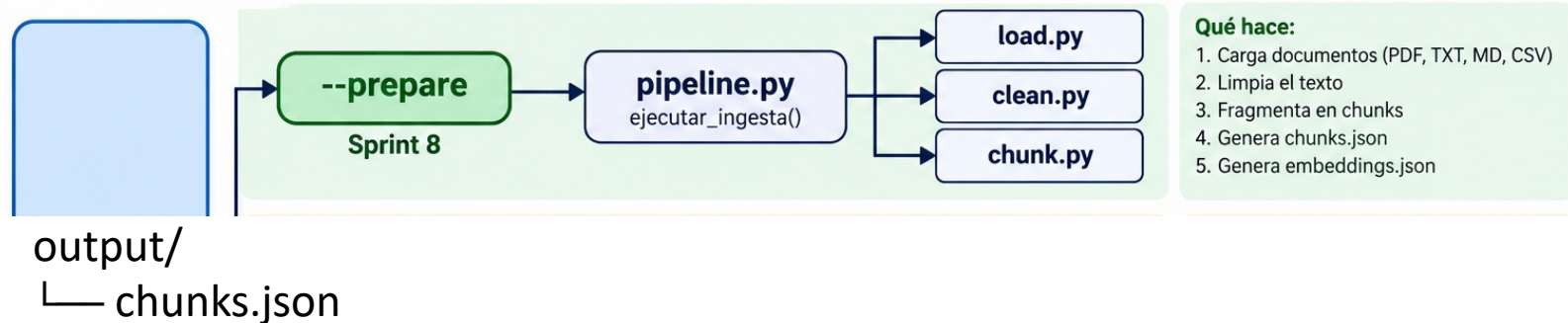
`CHUNK_OVERLAP = 100`



SPRINT 08 - SPRINT 09 -> SPRINT 10 : CÓDIGO

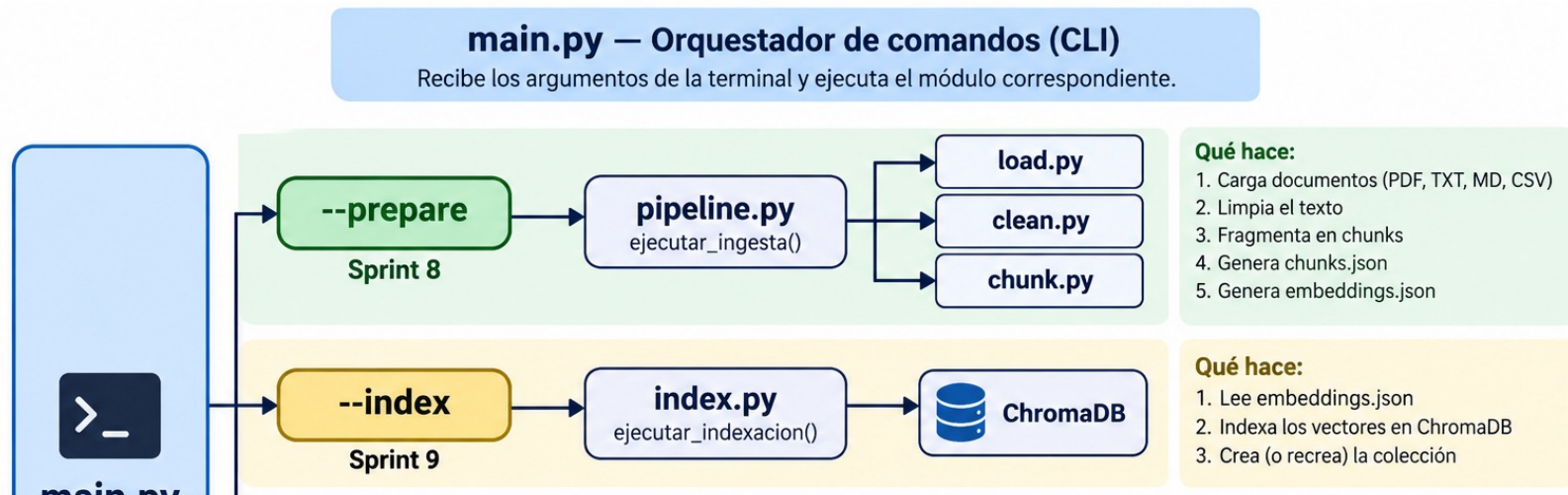
main.py — Orquestador de comandos (CLI)

Recibe los argumentos de la terminal y ejecuta el módulo correspondiente.



```
"chunks": [  
  {  
    "text": "Medición: Velocidad del viento (código 81)\nMunicipio: 102\nEstación:  
"metadata": {  
  "source": "/Users/luisspain/Documents/TheBridge/repos/Sprint_10/material_boot  
  "tipo": "meteo_medicion",  
  "magnitud": "81",  
  "municipio": "102",  
  "estacion": "1",  
  "punto_muestreo": "28102001_81_89",  
  "chunk_index": 0,  
  "chunk_size": 777  
}  
},  
],
```

SPRINT 08 - SPRINT 09 -> SPRINT 10 : CÓDIGO

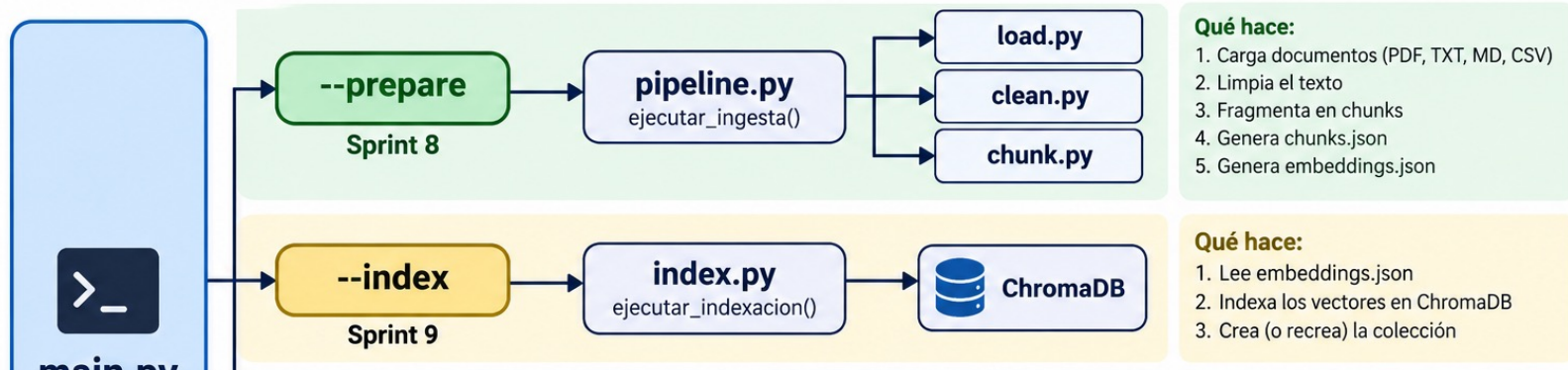


- Sprint 8 prepara y vectoriza el conocimiento.
- Sprint 9 convierte esos vectores en un índice consultable.

SPRINT 08 - SPRINT 09 -> SPRINT 10 : CÓDIGO

main.py — Orquestador de comandos (CLI)

Recibe los argumentos de la terminal y ejecuta el módulo correspondiente.



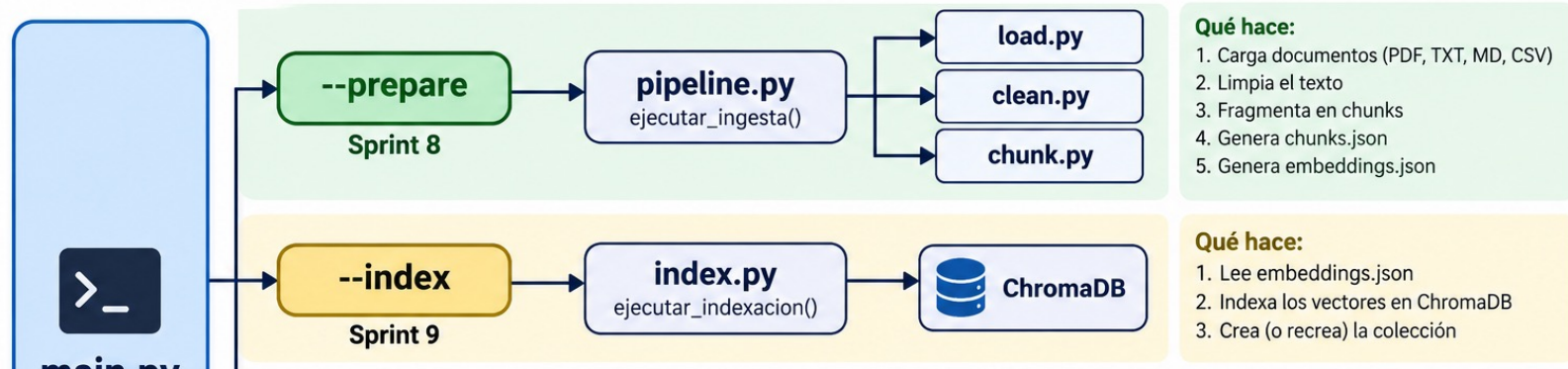
```
% python main.py --index
```

```
def _cmd_index(recreate: bool) -> None:
    total = ejecutar_indexacion(recreate=recreate)
    print(f"\nÍndice listo: {total} vectores en ChromaDB.")
```

SPRINT 08 - SPRINT 09 -> SPRINT 10 : CÓDIGO

main.py — Orquestador de comandos (CLI)

Recibe los argumentos de la terminal y ejecuta el módulo correspondiente.



```
% python main.py --index
```

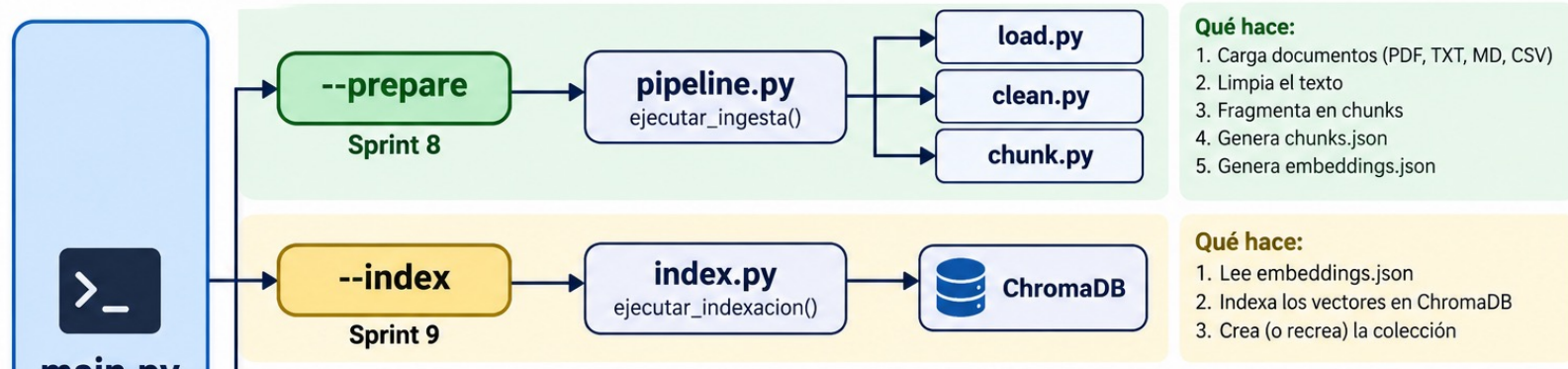
```
def _cmd_index(recreate: bool) -> None:
    total = ejecutar_indexacion(recreate=recreate)
    print(f"\nÍndice listo: {total} vectores en ChromaDB.")
```

```
Índice listo: 52 vectores en ChromaDB.
```

SPRINT 08 - SPRINT 09 -> SPRINT 10 : CÓDIGO

main.py — Orquestador de comandos (CLI)

Recibe los argumentos de la terminal y ejecuta el módulo correspondiente.



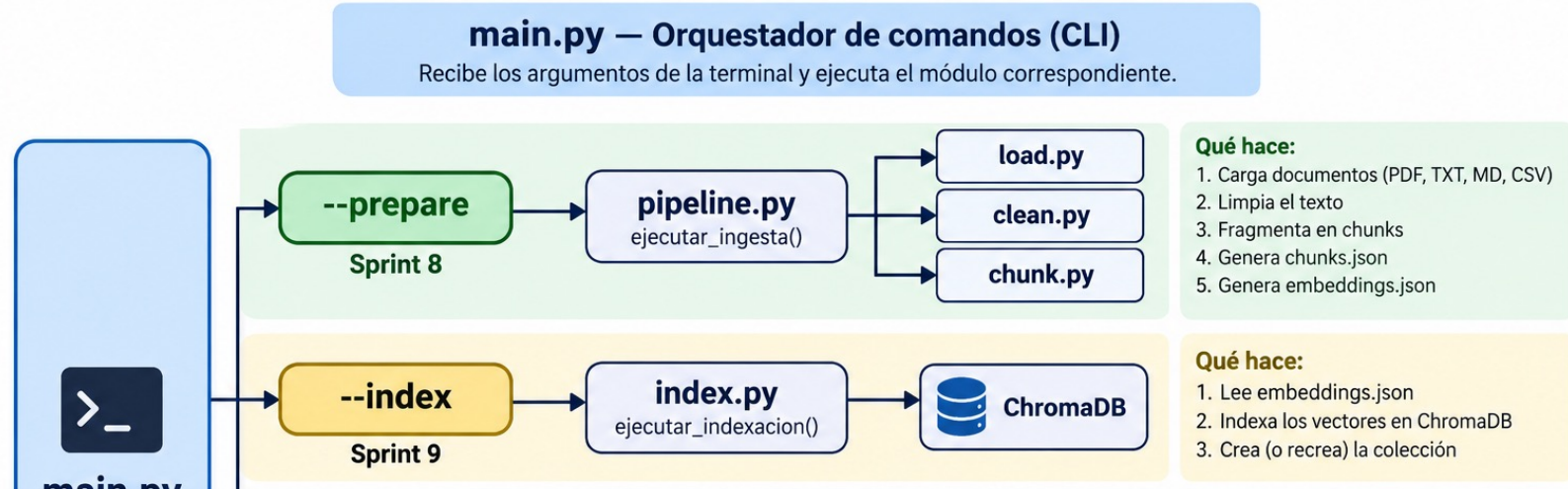
Crear la collection

```
...
def obtener_coleccion(
    client: chromadb.PersistentClient,
    crear: bool = True,
) -> chromadb.Collection:
    ...
```

collection.add()

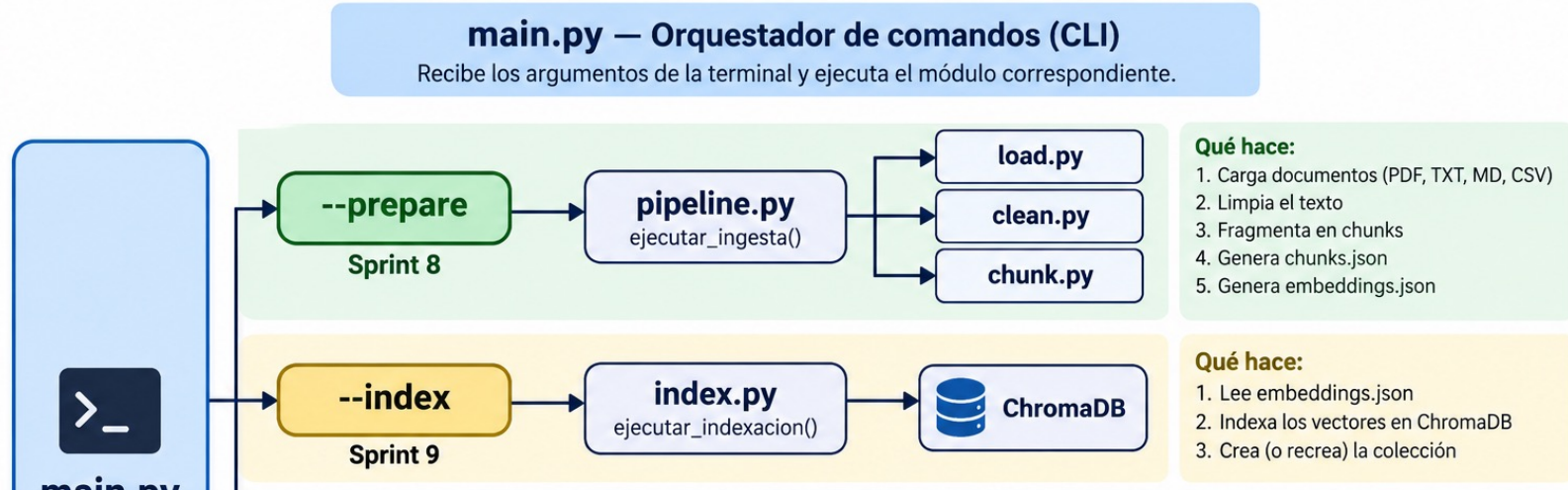
```
...
# 5) Insertar en lotes para no saturar memoria con corpora grandes
for inicio in range(0, len(ids), INDEX_BATCH_SIZE):
    fin = inicio + INDEX_BATCH_SIZE
    collection.add(
        ids=ids[inicio:fin],
        embeddings=embeddings[inicio:fin],
        documents=documents[inicio:fin],
        metadatas=metadatas[inicio:fin],
    )
...
```

SPRINT 08 - SPRINT 09 -> SPRINT 10 : CÓDIGO



Qué ocurre si vuelvo a ejecutar:
`python main.py --index`

SPRINT 08 - SPRINT 09 -> SPRINT 10 : CÓDIGO



Qué ocurre si vuelvo a ejecutar:
`python main.py --index`

`get_or_create_collection()`

SPRINT 08 - SPRINT 09 -> SPRINT 10 : CÓDIGO

Qué ocurre si vuelvo a ejecutar:

```
python main.py --index
```

```
get_or_create_collection()
```

la recupera.

Pero después el programa vuelve a ejecutar:

```
collection.add(...)
```

con los mismos IDs:

```
chunk_0
```

```
chunk_1
```

```
chunk_2
```

```
...
```

Eso puede producir conflictos porque esos IDs ya están en la colección.

SPRINT 08 - **SPRINT 09** -> SPRINT 10 : CÓDIGO

Por eso el proyecto tiene:

python main.py --index --recreate-index

En ese caso:

if recreate:

 borrar_coleccion(client)

y:

client.delete_collection(COLLECTION_NAME)

Después vuelve a:

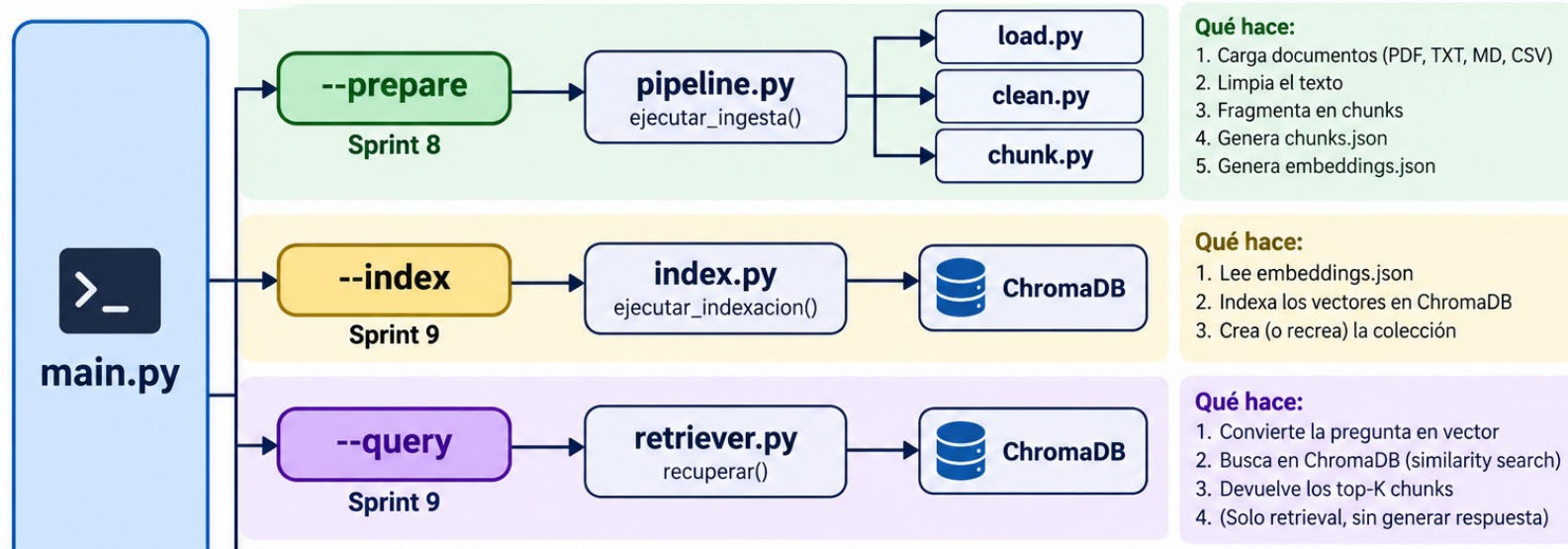
get_or_create_collection(...)

y crea una colección limpia.

SPRINT 08 - SPRINT 09 -> SPRINT 10 : CÓDIGO

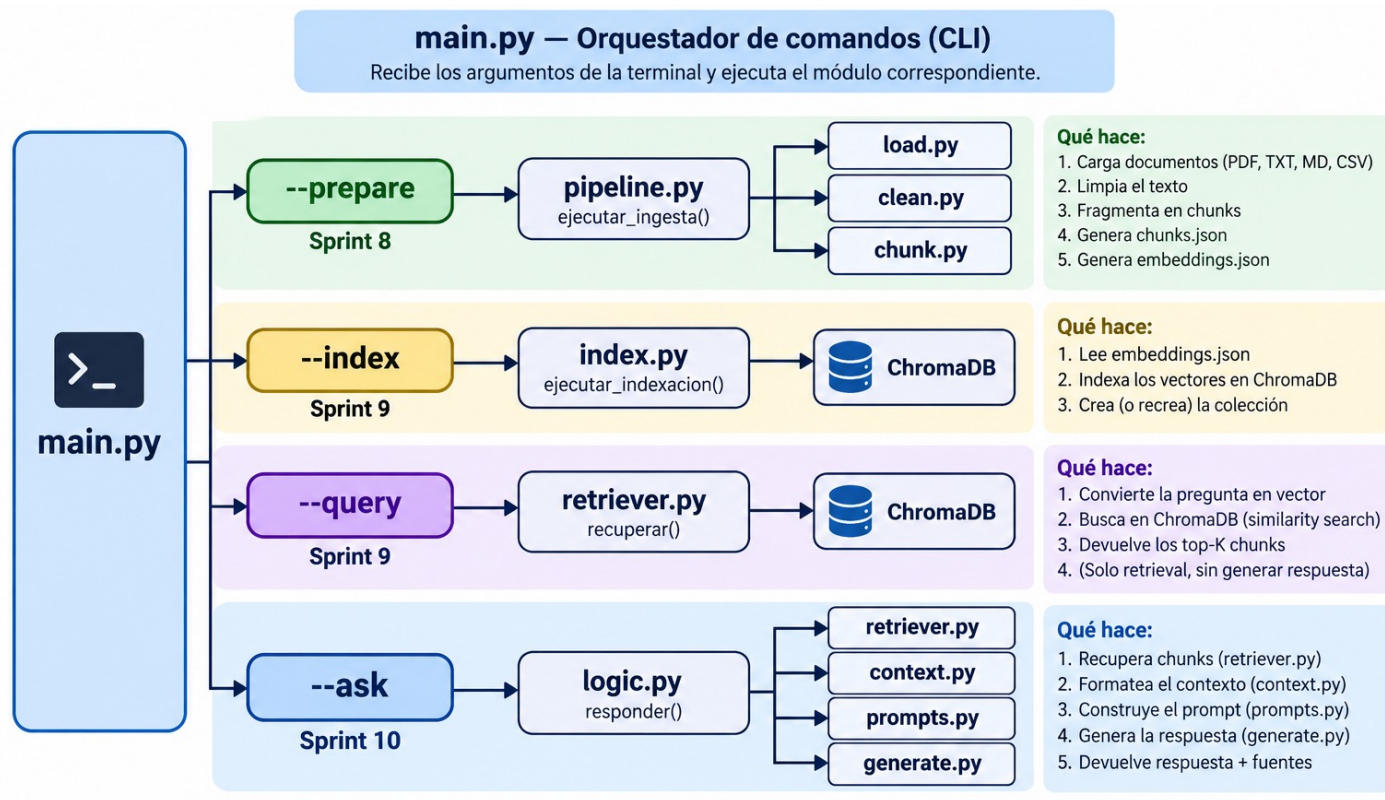
main.py — Orquestador de comandos (CLI)

Recibe los argumentos de la terminal y ejecuta el módulo correspondiente.



```
python main.py --query "¿Qué mide la magnitud 83?"
```

SPRINT 08 - SPRINT 09 -> **SPRINT 10** : CÓDIGO



```
python main.py --ask "¿Qué mide la magnitud 83?"
```

SPRINT 08 - SPRINT 09 -> SPRINT 10 : CÓDIGO

```
def responder(pregunta: str, top_k: int | None = None) -> dict:
    """Pipeline: retrieve → prompt → generate."""
    if not (pregunta or "").strip():
        return {
            "respuesta": "",
            "contexto": "",
            "chunks": [],
            "fuentes": [],
            "error": "La pregunta no puede estar vacía.",
        }

    chunks = recuperar(pregunta.strip(), top_k=top_k)
    contexto = formatear_contexto(chunks)
    prompt = build_rag_prompt(contexto, pregunta.strip())
    respuesta = generar_respuesta(prompt)

    return {
        "respuesta": respuesta,
        "contexto": contexto,
        "chunks": chunks,
        "fuentes": _extraer_fuentes(chunks),
        "error": None,
    }
```

SPRINT 08 - SPRINT 09 -> **SPRINT 10** : Streamlit

Desde el directorio donde se encuentra el archivo app.py, lanzar:

`streamlit run app.py`



Puede utilizar preguntas en queries:

Anexo - Comandos

No olvidar instalar las librerías en el entorno virtual:
pip install -r requirements.txt